

R basic (II)

Other types of objects

- **factor**
 - handle categorical data.
- **list**
 - a general form of vector in which the various elements need not be of the same type.
- **data frame**
 - matrix-like structures, in which the columns can be of different types.

Vector, Factor and List (I)

- Vector

```
> vec<-1:9
> vec
[1] 1 2 3 4 5 6 7 8 9
> is.vector(vec)
[1] TRUE
```

- Factor

```
> fac<-as.factor(vec)
> fac
[1] 1 2 3 4 5 6 7 8 9
Levels: 1 2 3 4 5 6 7 8 9
> is.vector(fac)
[1] FALSE
> is.factor(fac)
[1] TRUE
> levels(fac)
[1] "1" "2" "3" "4" "5" "6" "7" "8" "9"

> yy<-factor(c(1:7,5:9,8:3), levels=c(9:1))
> table(yy)
```

Vector, Factor and List (II)

```
>x<-c("B","F","A","C","A","C","B","A","F","D")  
>x1<-as.factor(x)  
>x2<-ordered(x, levels=c("B","F","A","C","D"))  
>x2[x2 >= "A"]  ## same with x2[which(x2 >= "A")]
```

Vector, Factor and List (III)

- List

```
> lis<-as.list(vec)
```

```
> lis
```

```
[[1]]  
[1] 1
```

```
[[2]]  
[1] 2
```

```
[[3]]  
[1] 3
```

```
[[4]]  
[1] 4
```

```
[[5]]  
[1] 5
```

```
[[6]]  
[1] 6
```

```
[[7]]  
[1] 7
```

```
[[8]]  
[1] 8
```

```
[[9]]  
[1] 9
```

```
>xx<-list("a", TRUE, c=1:3)
```

```
>xx[1:2]
```

```
>xx[[3]]
```

```
>xx[["c"]]
```

```
>xx$c
```

```
>ListEx <- list(name="Fred", wife="Mary",  
  no.children=3, child.ages=c(4,7,9))
```

```
>ListEx[4]          ## list
```

```
>ListEx[[4]]        ## vector
```

```
or ListEx$child.ages
```

```
>ListEx[[4]][1]
```

```
or ListEx$child.ages[1]
```

Vector, Factor and List (IV)

```
> str(vec)
int [1:9] 1 2 3 4 5 6 7 8 9
> str(fac)
Factor w/ 9 levels "1","2","3","4",...: 1 2 3 4 5 6 7 8 9
> str(lis)
List of 9
 $ : int 1
 $ : int 2
 $ : int 3
 $ : int 4
 $ : int 5
 $ : int 6
 $ : int 7
 $ : int 8
 $ : int 9
```

Matrix and Data Frames (I)

Data frame: ## a matrix but allow different mode for each column

```
> accounts<-data.frame(state=c("PA", "CA", "MA"), income=c(1000, 1100, 900), married=c(T, F, F))
```

```
> attributes(accounts)
```

```
> str(accounts)
```

```
> mat<-matrix(data=1:35, nrow=5, ncol=7)
```

```
> dat<-as.data.frame(mat)
```

```
> mat
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]
[1,]	1	6	11	16	21	26	31
[2,]	2	7	12	17	22	27	32
[3,]	3	8	13	18	23	28	33
[4,]	4	9	14	19	24	29	34
[5,]	5	10	15	20	25	30	35

```
> dat
```

	V1	V2	V3	V4	V5	V6	V7
1	1	6	11	16	21	26	31
2	2	7	12	17	22	27	32
3	3	8	13	18	23	28	33
4	4	9	14	19	24	29	34
5	5	10	15	20	25	30	35

Matrix and Data Frames (II)

```
> str(dat)
'data.frame':   5 obs. of  7 variables:
 $ V1: int  1 2 3 4 5
 $ V2: int  6 7 8 9 10
 $ V3: int 11 12 13 14 15
 $ V4: int 16 17 18 19 20
 $ V5: int 21 22 23 24 25
 $ V6: int 26 27 28 29 30
 $ V7: int 31 32 33 34 35

> dat$V1
[1] 1 2 3 4 5
> dat$V2
[1] 6 7 8 9 10
> dat[,1]
[1] 1 2 3 4 5
> dat[,2]
[1] 6 7 8 9 10

> str(mat)
int [1:5, 1:7] 1 2 3 4 5 6 7 8 9 10 ...
> mat[,1]
[1] 1 2 3 4 5
> mat[,2]
[1] 6 7 8 9 10
> colnames(mat)
NULL
> colnames(mat)<-paste("V",1:dim(mat)[2],sep="")
> colnames(mat)
[1] "V1" "V2" "V3" "V4" "V5" "V6" "V7"
> mat$V1
錯誤在mat$V1 : $ operator is invalid for atomic vectors
> mat[, "V5"]
[1] 21 22 23 24 25
```


Example (I)

- Edgar Anderson's Iris Data

- data set in the R base package
- 150obs, 5 variables

>?iris

- Call and survey data

>data(iris)

>iris #data frame

>fix(iris)

>str(iris)

- Select elements, columns, or rows in data frame

>iris[1,1]

>iris[2,]

>iris[,3]

>iris[,3:5]

>iris[c(1,9,150),]

>iris[iris[,1]>6,]

Example (II)

- Example of “array”

```
>data(iris3)
```

```
>iris3          #3-dim array(50*4*3)
```

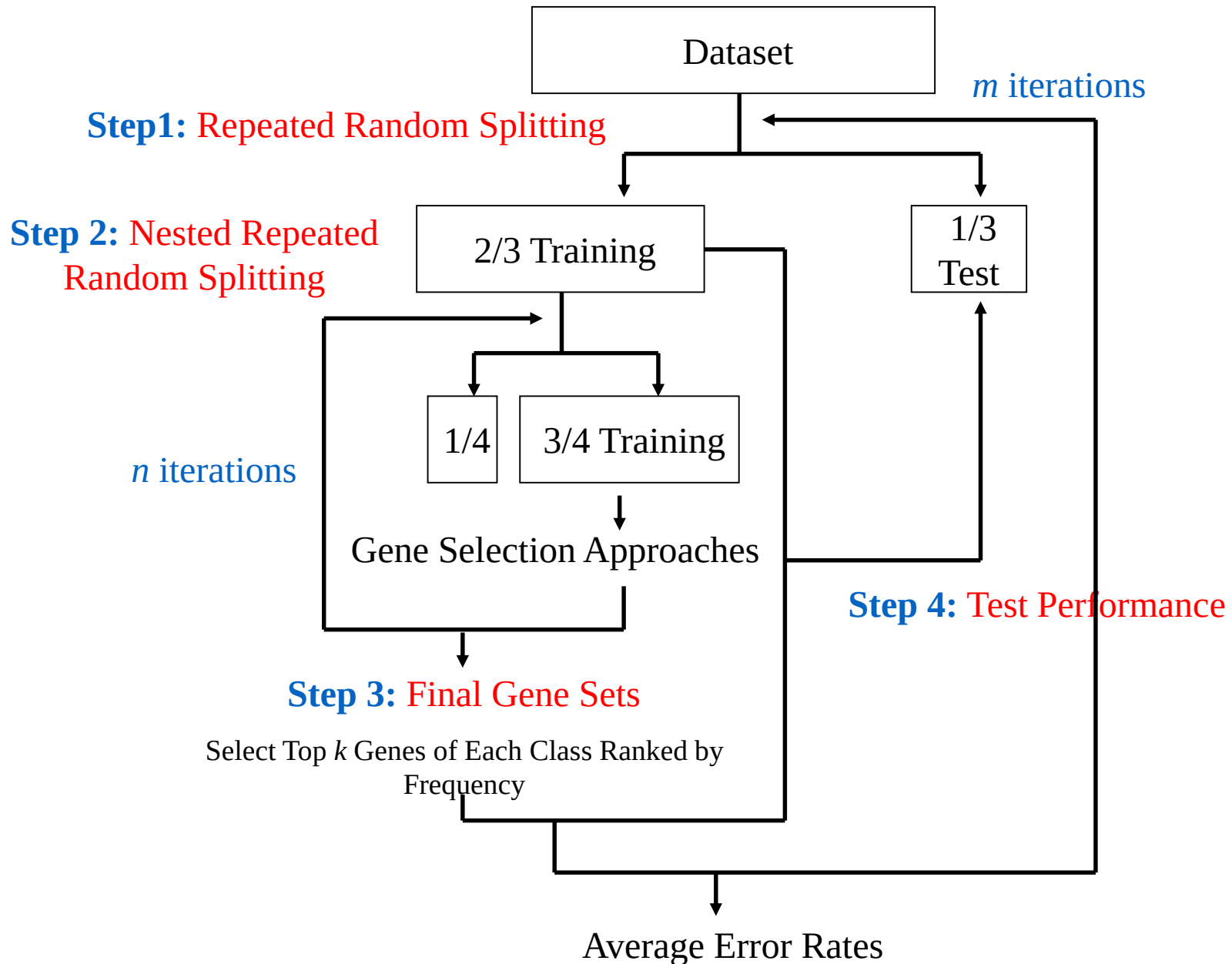
```
>class(iris3)
```

```
>iris3[1:5, ,3]
```

```
>class(iris3[1:5, ,3])
```

```
>iris3[1:3,2:3, ]
```

For a N -class problem

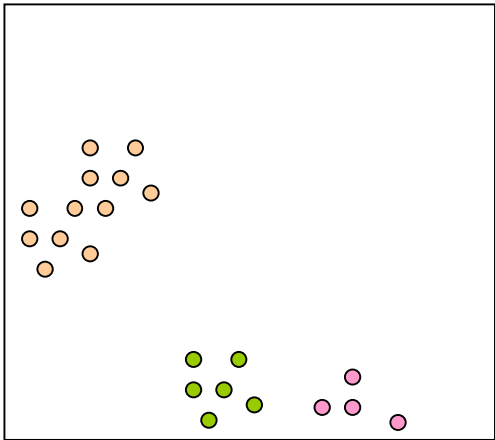


Gene Selection Strategy: Frequency-based Method

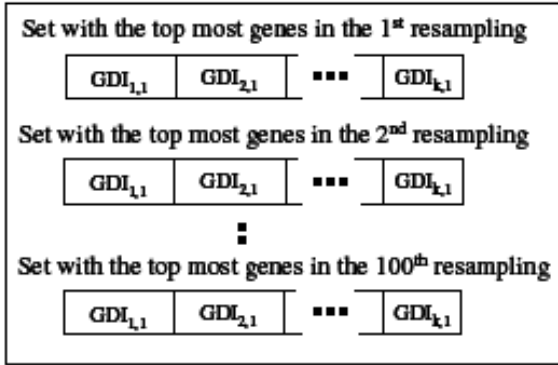
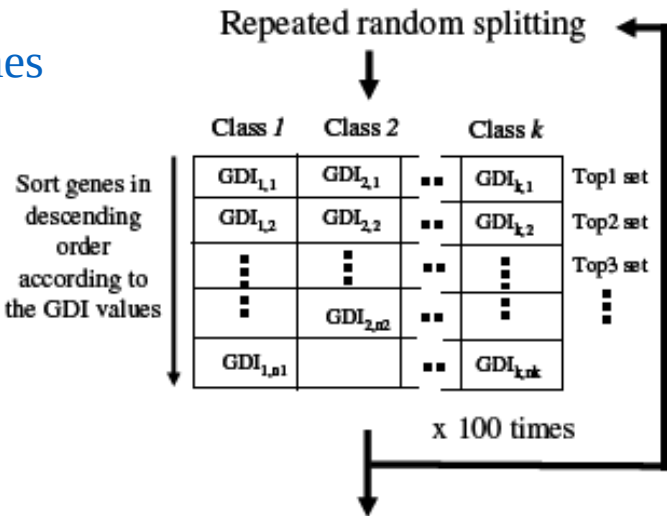
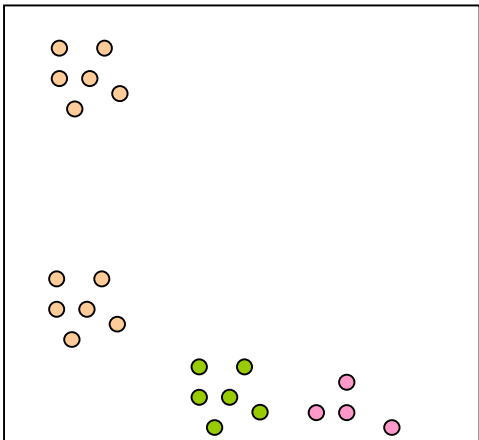
For a k -class problem 2/3 samples used for gene selection

Repeated random splitting 100 times

GDI=0.6



GDI=0.7



With highest GDI value (or top N genes)

Compute the frequency with which a gene appears in the list of dominant (dormant) genes

Select dominant (dormant) genes with frequency $\geq T$

Test performance

Best m Marker Genes for Each Class

SRBCT Dominant Genes											
EWS			BL			NB			RMS		
rownames	frequency	ave.GDI	rownames	frequency	ave.GDI	rownames	frequency	ave.GDI	rownames	frequency	ave.GDI
770394	85	1.91	236282	64	2.21	812105	68	1.73	784224	86	1.71
377461	7	1.75	745019	9	2.41	325182	10	1.62	796258	8	1.47
814260	5	1.69	1469292	6	2.61	134748	7	1.59			
			297392	5	2.37	756401	5	1.65			

- **Top 5 genes** for each class (100 data splitting experiments)

SRBCT Dominant Genes											
EWS			BL			NB			RMS		
rownames	frequency	ave. GDI	rownames	frequency	ave. GDI	rownames	frequency	ave. GDI	rownames	frequency	ave. GDI
770394	100	1.88	236282	100	2.19	812105	99	1.65	784224	100	1.60
814260	100	1.43	183337	67	1.82	134748	64	1.45	796258	96	1.27
377461	99	1.46	745019	51	2.03	756401	56	1.41	244618	65	1.18
1435862	94	1.37	1469292	24	1.90	325182	33	1.38	839552	42	1.14
866702	88	1.28	47475	24	1.85	629896	32	1.32	769716	38	1.12
1471841	10	1.29	767183	19	2.03	486110	26	1.35	898219	25	1.11
491565	5	1.24	624360	19	1.80	786084	26	1.34	789253	22	1.12
			814526	18	1.73	810864	17	1.27	298062	21	1.11
			769657	18	1.60	81518	15	1.26	25725	14	1.10
			868304	16	1.97	823886	11	1.33	296448	8	1.15
			68977	16	1.78	770675	10	1.50	143306	8	1.13
			840942	12	1.83	544664	9	1.45	246035	8	1.06
			241412	12	1.70	383188	9	1.36	1409509	6	1.12
			855487	11	1.94	244637	8	1.26	42558	6	1.07
			490772	10	1.62	950574	7	1.23	207274	5	1.23
			785793	9	1.81	789376	7	1.22	293500	5	1.15
			701751	7	1.71	823775	6	1.25	142134	5	1.10
			626502	7	1.71				813841	5	1.09

Example (III)

```
> rfac<-sample(9,20,replace=T)
> rfac
[1] 9 7 5 2 1 2 9 8 7 3 7 3 7 9 2 6 5 4 4 2
> rfac<-factor(rfac,labels=letters[1:9])
> rfac
[1] i g e b a b i h g c g c g i b f e d d b
Levels: a b c d e f g h i
> summary(rfac)
 a b c d e f g h i
1 4 2 2 2 1 4 1 3

> rvec<-sample(9,20,replace=T)
> rvec
[1] 3 2 2 1 9 7 6 5 4 8 2 9 6 3 1 5 1 4 3 2
> summary(rvec)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   1.00   2.00   3.50   4.15   6.00   9.00
```

Exercise (I)

```
> n100<-sample(1:9,100,replace=T)
> n100
 [1] 4 8 5 9 7 3 9 3 4 4 4 3 4 3 1 5 8 4 5 8 7 5 8 3 4
[76] 3 1 8 1 3 9 3 1 6 2 5 1 2 8 3 6 3 3 9 7 3 6 9 9 4
```

- What are the top 3 numbers with highest frequencies?
 - Please provide the number and corresponding frequency

Hint: summary, sort

Answers to Exercise I

```
> as.factor(n100)
 [1] 4 8 5 9 7 3 9 3 4 4 4 3 4 3 1 5 8 4 5 8 7 5 8 3 4 1 7 3 8
[46] 2 1 9 2 3 1 8 3 3 8 3 6 1 4 8 8 9 3 1 1 6 7 5 1 1 1 6 9 7
[91] 6 3 3 9 7 3 6 9 9 4
Levels: 1 2 3 4 5 6 7 8 9
> summary(as.factor(n100))
 1  2  3  4  5  6  7  8  9
18  7 20 11  8  6  6 14 10
> sort(summary(as.factor(n100)))
 6  7  2  5  9  4  8  1  3
 6  6  7  8 10 11 14 18 20
> sort(summary(as.factor(n100)),decreasing=T)
 3  1  8  4  9  5  2  6  7
20 18 14 11 10  8  7  6  6
> sort(summary(as.factor(n100)),decreasing=T)[1:3]
 3  1  8
20 18 14
> as.numeric(sort(summary(as.factor(n100)),decreasing=T)[1:3])
[1] 20 18 14
> names(sort(summary(as.factor(n100)),decreasing=T)[1:3])
[1] "3" "1" "8"
```


Exercise (II)

```
> for(i in 1:10){  
+   print(sample(9,4,replace=T))  
+ }  
[1] 2 5 5 2  
[1] 6 2 1 5  
[1] 9 5 9 7  
[1] 3 1 8 7  
[1] 8 2 5 7  
[1] 5 9 3 9  
[1] 4 7 1 4  
[1] 7 5 7 6  
[1] 7 2 2 1  
[1] 4 7 6 7
```

- Please repeat this procedure for 100 time
 - Record those 4 values of each time
 - Show the top 3 numbers with highest frequencies for each position

Hint: `list100<-vector(mode="list", length=4)`
`list100[[1]]<-c(list100[[1]], position_1)`

Answers to Exercise II-1

```
> lis100<-vector(mode="list",length=4)
> for(i in 1:100){
+   rand4<-sample(9,4,replace=T)
+   for(j in 1:length(rand4)){
+     lis100[[j]]<-c(lis100[[j]],rand4[j])
+   }
+ }
> lis100
[[1]]
  [1] 3 6 3 9 9 3 1 9 9 4 2 5 5 5 2 2 7 6 4 9 5
 [46] 9 9 7 7 1 4 4 6 5 9 2 6 1 6 7 1 5 1 6 4 1
 [91] 3 8 5 8 4 7 4 9 8 9

[[2]]
  [1] 8 8 9 6 8 6 7 9 4 8 9 1 5 7 3 8 7 1 7 5 9
 [46] 4 9 8 3 8 1 2 9 6 8 7 2 7 7 7 9 1 8 9 2 1
 [91] 5 4 1 2 7 2 3 8 3 5

[[3]]
  [1] 7 9 6 3 9 7 4 7 7 8 8 1 5 7 5 7 5 9 5 5 2
 [46] 1 9 3 4 1 1 1 9 3 5 3 1 6 2 7 5 7 2 8 1 4
 [91] 7 7 1 8 1 8 4 2 4 3

[[4]]
  [1] 4 5 3 3 2 3 8 4 8 2 2 8 4 6 6 5 6 2 5 8 7
 [46] 7 7 2 3 1 9 9 4 3 9 8 6 6 4 3 2 8 9 8 6 9
 [91] 9 7 4 6 6 6 8 7 2 3
```

Answers to Exercise II-2

```
> for(i in 1:length(lis100)){  
+   print(sort(summary(factor(lis100[[i]])),decreasing=T)[1:3])  
+ }  
 1  5  9  
14 14 14  
 8  7  9  
17 14 13  
 4  7  1  
15 14 12  
 6  8  2  
16 13 12
```

自訂函數

`user_defined_function.R`

R 可以將常重複執行的程式碼定義成函數，如以下定義

```
函數名稱 <- function(參數){  
  程式重複執行的部份  
}
```

Apply Function

```
> x <- cbind(x1 = 3, x2 = c(4:1, 2:5))  
> col.sums <- apply(x, 2, sum)  
> row.sums <- apply(x, 1, sum)
```

Apply Solution for HW1

```
> aa <- read.delim("SRBCT_train.txt")  
> bb <- as.matrix(aa[,-c(1,2)])  
> mean_gene <- apply(bb, 1, mean)  
> mean_sample <- apply(bb, 2, mean)
```

Import / Export data (I)

- Write data to .txt file

```
>write.table(iris, "iris.txt")
```

```
>?write.table
```

```
>write.table(iris,"iris2.txt",quote=F, row.names=F)
```

```
> write.table(iris,"iris3.txt",quote=F, row.names=F, col.names=F)
```

- Read data from .txt file

```
>demodata<-read.table("iris.txt")
```

```
>demodata<-read.delim("iris.txt")
```

```
>fix(demodata)
```

- R can read data stored by S, SAS, SPSS by using functions in the package "foreign".
- read "R Data Import/Export" manual

Import / Export data (II)

- import several .txt files :

```
>files.n<-list.files(path="D:/tmp/", ".txt", full.names=T)
```

```
# return full names of all .txt file in the directory. ( a character vector ),  
ex: "d:/datas/test1.txt", "d:/datas/test2.txt",...
```

```
>files.l<-lapply(files.n,read.table,header=T)
```

```
# type of return object is "list".
```

```
# ex: files.l[[1]]:test1.txt, files.l[[2]]:test2.txt, ...
```

Import / Export data (III)

```
>directory<-"D:/tmp/"
>aa<-read.table(paste(directory, "test_data.txt", sep=""))
>aa<-cbind(aa, garage=c(1,0,2,1, 2))
>write.table(aa, paste(directory, "newData.txt", sep=""))
>write.table(aa, paste(directory, "newData1.txt", sep=""),
  sep="\t", quote=F)
>write.table(aa, paste(directory, "newData2.txt", sep=""),
  sep="\t", quote=F, row.names=F, col.names=F)
>bb<-read.table(paste(directory, "newData2.txt", sep=""))
>row.names(aa)
>row.names(bb)
>names(aa)
>names(bb)
```

It is better to use “read.delim”

HW2

- For the file “cycleData_peak.txt”
 - Count how many not “NA” in each row.
- For the file “SRBCT_train.txt”.
 - Normalize data by each row (let values b/w 0~1)
 - Normalize data by each column (mean = 0, sd = 1)

Hint:

```
nor<-function(x){  
  min.x<-min(x)  
  max.x<-max(x)  
  (x-min.x)/(max.x-min.x)  
}
```

```
NA.counting<-function(x){  
  N<-length(x)  
  n<-sum(is.na(x))  
  M<-N-n  
  return(M)  
}
```